TECHNICAL ARCHITECTURE & TESTING MANUAL

System Integration & Testing Architecture

Cross-Repository Pipeline: Linking the 3D Digital Twin, AnyLogic Multi-Method Engine, RL Supervisors & Streamlit Dashboard

PRIMARY REPOSITORIES
AI-3D-Robotics-Lab & mdc-robotics-simulation-twin

COURSE
Simulation for Applied AI (MDC)

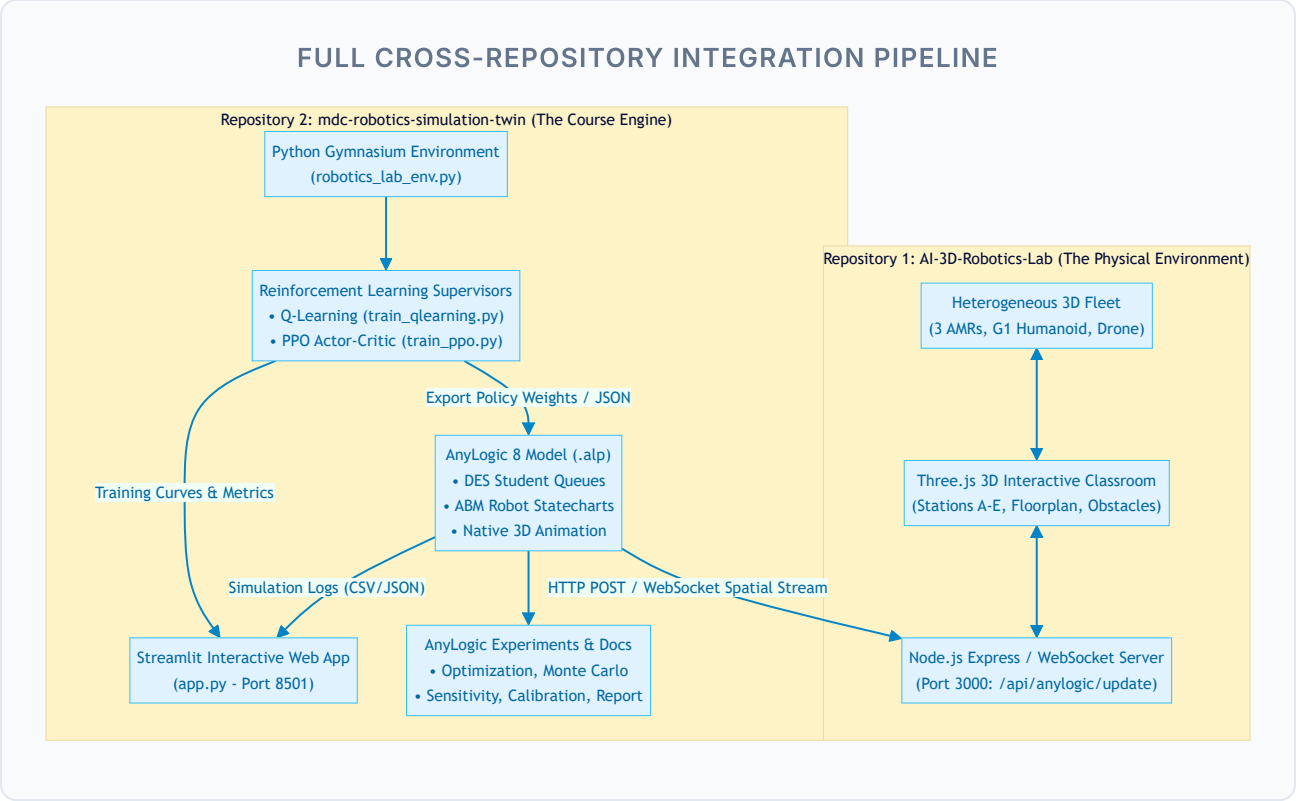
AUTHOR & LEAD
Jorge A Mendoza

TEAM MEMBERS
Jordan, Illia, Ricardo

1. Executive Integration Overview

The system architecture separates the project cleanly into two complementary tiers:

- **Physical & Spatial Tier (`AI-3D-Robotics-Lab`):** Hosts the Three.js 3D classroom CAD environment, 360° rotating LiDAR obstacle sensors, draggable laboratory stations (A through E), and the Node.js WebSocket/HTTP bridge.
- **Analytical & Simulation Tier (`mdc-robotics-simulation-twin`):** Hosts the course deliverables including the AnyLogic multi-method model (DES queueing + ABM statecharts), the Python Reinforcement Learning dispatch policies (Q-Learning vs. PPO), and the multi-tab Streamlit dashboard.



2. What Each Team Member Needs to Know About the Environment

To ensure each team member can work independently without roadblocks, here are the essential environment specifications for each role:

🧠 For Jordan Gonzalez (Reinforcement Learning Lead)

Folder: `/rl_models` | **Runtime:** Python 3.10+ (Standard library, NumPy, Gymnasium)

- **Observation Space (8 Dimensions):** `[Q_A, Q_B, Q_C, B_1, B_2, B_3, Active_Docks, Demand_Phase]`
  - `Q_A, Q_B, Q_C`: Queue counts at Parts Shelf, Soldering Bench, and QA Testing (0 to 20).
  - `B_1, B_2, B_3`: Battery levels of AMR-01, AMR-02, AMR-03 (0.0 to 1.0).
  - `Active_Docks`: Number of AMRs currently occupying Station D charging pads (0 to 2).
  - `Demand_Phase`: Multiplier representing class rush hour intensity (0.5 to 1.5).
- **Action Space (5 Discrete Actions):** `0`: Dispatch AMR to Station A | `1`: Dispatch to Station B | `2`: Dispatch to Station C | `3`: Send lowest-battery AMR to Station D (Charging Dock) | `4`: Standby / Idle.
- **Episode Length:** 120 steps (represents a 2-hour lab class session at 1 minute per step).
- **Deliverable for Dashboard:** Ensure running `train_qlearning.py` and `train_ppo.py` generates `q_learning_results.json` and `ppo_results.json`.

📊 For Illia Shybanov (Dashboard & UI Lead)

Folder: `/dashboard` | **Runtime:** Python (Streamlit, Plotly, Pandas)

- **Launch Command:** `streamlit run app.py` (runs locally on `http://localhost:8501`).
- **Data Ingestion:** The dashboard automatically parses results from `../rl_models/q_learning_results.json` and `../rl_models/ppo_results.json`.
- **Interactive Controls:** Sliders for fleet size (1–6), student request influx (5–40 req/hr), and transit speed (0.5–2.0 m/s) dynamically re-calculate performance KPIs.
- **5 Required Tabs:** Operational KPIs, AI Policy Comparison (RL), Simulation Experiments, 3D Digital Twin Status, and Executive Report.

🔬 For Ricardo Molina Gonzalez (Experiments & Documentation Lead)

Folder: `/docs` | **Runtime:** AnyLogic 8 (PLE or University) & Word/Markdown

- **AnyLogic Experiments Guide:** `docs/ANYLOGIC_EXPERIMENTS_GUIDE.md` contains the exact click paths to configure the 5 required AnyLogic experiments:
  1. **Optimization:** Find optimal AMR count (3 AMRs) vs. charging pads (2 docks).
  2. **Compare Runs:** FIFO Baseline vs. Q-Learning vs. PPO.
  3. **Monte Carlo:** 100 replications with stochastic arrival intervals (95% CI).
  4. **Sensitivity Analysis:** Varying battery drain rate (±30%) and student influx.
  5. **Calibration:** Calibrating simulated transit speeds against MDC stopwatch data.
- **Deliverable:** Export experiment charts/data tables and place them into the `FINAL_PROJECT_REPORT_TEMPLATE.md` and presentation slides.

🏗️ For Jorge A Mendoza (AnyLogic Model Architect & Team Lead)

Folder: `/anylogic_model` | **Runtime:** AnyLogic 8 + Node.js (Port 3000)

- **Model Architecture:** Multi-method connecting Process Modeling Library (DES) blocks to AMR agent statecharts (ABM).
- **Spatial Geometry:** Uses `assets/robotics_lab_floorplan.png` scaled at 1m = 50px ($14.5 ext{m} imes 10.2 ext{m}$).
- **Live Bridge:** AnyLogic triggers HTTP POST updates to `http://localhost:3000/api/anylogic/update` with current robot coordinates.

3. End-to-End System Testing Protocol

When the team is ready to test the entire ecosystem together, follow this 3-terminal sequence:

Terminal 1: Start 3D Digital Twin Server (Visual Layer)

```
cd "D:\Miami Dade College\AI-3D-Robotics-Lab\digital_twin_app"
npm install
npm start
# Output: Server running on http://localhost:3000
```

Terminal 2: Run & Verify RL Policies (Decision Layer)

```
cd "D:\Miami Dade College\Simulation For AI\mdc_robotics_simulation_twin
l_models"
python train_qlearning.py
python train_ppo.py
# Output: Saves q_learning_results.json, ppo_results.json, and q_table.npy
```

Terminal 3: Launch Streamlit Dashboard (Presentation Layer)

```
cd "D:\Miami Dade College\Simulation For AI\mdc_robotics_simulation_twin\
pip install -r requirements.txt
streamlit run app.py
# Output: Dashboard opens in browser at http://localhost:8501
```

Step 4: Launch AnyLogic Simulation Model (Engine Layer)

Open `anylogic_model/MDC_Robotics_Lab_Simulation.alp` in AnyLogic 8, click **Run**, and verify that the DES queues clear dynamically while AMR agents visually navigate the 3D laboratory canvas.

4. Summary of Key Repository Interconnections

Source Component	Target Component	Protocol / Data Format	What is Transferred
Gymnasium RL (Jordan)	**AnyLogic Model (Jorge)**	Policy Rules / Lookup Table	Optimal action logic: When to dispatch to Station A/B/C vs. recharge at Station D.
AnyLogic Model (Jorge)	**3D Twin Server (Jorge)**	HTTP POST / WebSocket	Live (X, Y) robot positions, battery voltages, and station statuses.
RL Models (Jordan)	**Streamlit App (Illia)**	JSON (`q_learning_results.json`)	Cumulative rewards, training loss curves, and policy comparison tables.
AnyLogic Experiments (Ricardo)	**Streamlit & Report (Illia & Ricardo)**	CSV / Plotly Charts	Monte Carlo confidence intervals, Pareto optimization curve, and sensitivity plots.